

LeetCode

Problems

Author
Srinivas Dande





Java Learning Center

No.1 In Java Training & placement

344. Reverse String

Description:

- ✓ Write a function that reverses a string.
- ✓ The input string is given as an array of characters **chArr**.
- ✓ You must do this by modifying the input array in-place with $O(1)$ extra memory.

Example 1:

Input: s = ["h","e","l","l","o"]

Output: ["o","l","l","e","h"]

Example 2:

Input: s = ["H","a","n","n","a","h"]

Output: ["h","a","n","n","a","H"]

Constraints:

- $1 \leq s.length \leq 105$
- $s[i]$ is a printable ascii character.

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple

5

Microsoft

2

Amazon

2

Facebook

2

Google

2

Arista Networks

2



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Adobe **7**

tcs **3**

intel **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Goldman Sachs **3**

Oracle **3**

Cisco **2**

Paypal **2**

Bloomberg **2**

Spotify **2**

Cognizant **2**

344.Reverse String - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public void reverseString(char chArr[]) {

        int start=0;
        int end=chArr.length-1;

        while(start<end) {
            char temp = chArr[start];
            chArr[start]=chArr[end];
            chArr[end]=temp;

            start++;
            end--;
        }
    }
}
```

345. Reverse Vowels of a String

Description:

- ✓ Given a string str, reverse only all the vowels in the string and return it.
- ✓ The vowels are 'a', 'e', 'i', 'o', and 'u', and they can appear in both lower and upper cases, more than once.

Example 1:

Input: str = "hello"

Output: "holle"

Example 2:

Input: str = "leetcode"

Output: "leotcede"

Constraints:

- c) $1 \leq s.length \leq 3 * 10^5$
- d) s consist of printable ASCII characters.

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple 3



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Bloomberg 3

Yahoo 3

Uber 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Google 4

Facebook 4

Amazon 4

Adobe 2

345. Reverse Vowels of a String - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class Solution {
    public boolean isVowel(char c) {
        return c == 'a' || c == 'i' || c == 'e' || c == 'o' || c == 'u' || c == 'A' || c
            == 'I' || c == 'E' || c == 'O' || c == 'U';
    }

    public String reverseVowels(String str) {

        int n = str.length();
        char chArr[] = str.toCharArray();

        int start = 0;
        int end = chArr.length - 1;
```



```
while (start < end) {  
  
    while (start < n && !isVowel(chArr[start])) {  
        start++;  
    }  
  
    while (end >= 0 && !isVowel(chArr[end])) {  
        end--;  
    }  
  
    if (start < end) {  
        char temp = chArr[start];  
        chArr[start] = chArr[end];  
        chArr[end] = temp;  
    }  
  
    start++;  
    end--;  
  
}  
String resultStr = new String(chArr);  
return resultStr;  
  
}
```



151. Reverse Words in a String

Description:

- ✓ Given an input string **str**, reverse the order of the words.
- ✓ A word is defined as a sequence of non-space characters. The words in **str** will be separated by at least one space.
- ✓ Return a string of the words in reverse order concatenated by a single space.

Note that **str** may contain leading or trailing spaces or multiple spaces between two words. The returned string should only have a single space separating the words. Do not include any extra spaces.

Example 1:

Input: str = "the sky is blue"

Output: "blue is sky the"

Example 2:

Input: str = " hello world "

Output: "world hello"

Explanation: Your reversed string should not contain leading or trailing spaces.

Example 3:

Input: str = "a good example"

Output: "example good a"

Explanation: You need to reduce multiple spaces between two words to a single space in the reversed string.

Constraints:

- 1 <= s.length <= 104
- str contains English letters (upper-case and lower-case), digits, and spaces "
- c) There is at least one word in str.

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Apple **5** Microsoft **4** Amazon **2** Bloomberg **2**

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Google **4** VMware **3** Visa **3** Facebook **2** Uber **2** Nvidia **2**

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

LinkedIn **5** Salesforce **4** Oracle **4** Cisco **3** ByteDance **3**
Paypal **2** Citrix **2** Qualcomm **2** Adobe **2** PayTM **2**
Deutsche Bank **2** Cognizant **2** Snapchat Yelp

151. Reverse Words in a String - Solution

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
**/
```

```
class Solution {
```

```
    public String trimSpaces(String s) {
```

```
        int left = 0, right = s.length() - 1;
```

```
        while (left <= right && s.charAt(left) == ' '){
```

```
            ++left;
```

```
        }
```



```
while (left <= right && s.charAt(right) == ' '){
    --right;
}

StringBuilder sb = new StringBuilder();
while (left <= right) {
    char c = s.charAt(left);

    if (c != ' ') sb.append(c);
    else if (sb.charAt(sb.length() - 1) != ' ') sb.append(c);

    ++left;
}
return sb.toString();
}
```

public void reverse(char chArr[], int start, int end) {

```
    while (start < end) {
        char temp = chArr[start];
        chArr[start] = chArr[end];
        chArr[end] = temp;

        start++;
        end--;
    }
}
```

public String reverseWords(String str) {

```
    str=trimSpaces(str);
    char chArr[] = str.toCharArray();
    int n = chArr.length;

    int start = 0;
    int end=0;
```



```
        for (; end < n; end++) {  
            if (chArr[end] == ' ') {  
                reverse(chArr, start, end - 1);  
                start = end + 1;  
            }  
        }  
  
        reverse(chArr, start, end - 1);  
        reverse(chArr, 0, n - 1);  
  
        String revStr = new String(chArr);  
        return revStr;  
    }  
}
```



125. Valid Palindrome

Description:

- ✓ A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.
- ✓ Given a string str, return true if it is a palindrome, or false otherwise.

Example 1:

Input: str = "A man, a plan, a canal: Panama"

Output: true

Explanation: "amanaplanacanalpanama" is a palindrome.

Example 2:

Input: str = "race a car"

Output: false

Explanation: "raceacar" is not a palindrome.

Example 3:

Input: str = ""

Output: true

Explanation: str is an empty string "" after removing non-alphanumeric characters. Since an empty string reads the same forward and backward, it is a palindrome.

Constraints:

- a) $1 \leq \text{str.length} \leq 2 * 10^5$
- b) s consists only of printable ASCII characters.

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Facebook 15

Adobe 5

Apple 4

Google 4

American Express 2

Spotify 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon 10

Microsoft 9

Bloomberg 5

LinkedIn 5

Wayfair 3

Yandex 2

ServiceNow 2

TikTok 2

Infosys 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Cisco 3

Goldman Sachs 2

JPMorgan 2

Uber

Zenefits

125. Valid Palindrome - Solution

```

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class Solution {
    public boolean isPalindrome(String str) {
        str = str.toLowerCase();
        StringBuilder sb=new StringBuilder();
    }
}

```



```
for(int i=0;i<str.length();i++) {  
  
    char ch= str.charAt(i);  
    if( Character.isLetterOrDigit(ch)) {  
        sb.append(ch);  
    }  
  
}  
  
str=sb.toString();  
  
int start = 0;  
int end = str.length() - 1;  
  
while (start < end) {  
    if (str.charAt(start) != str.charAt(end)) {  
        return false;  
    }  
    start++;  
    end--;  
}  
  
return true;  
}  
  
}
```

242. Valid Anagram

Description:

- ✓ Given two strings main and str, return true if t is an anagram of s, and false otherwise.
- ✓ An Anagram is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all the original letters exactly once.

Example 1:

Input: main = "anagram", str = "nagaram"

Output: true

Example 2:

Input: main = "rat", str = "car"

Output: false

Constraints:

- $1 \leq s.length, t.length \leq 5 * 10^4$
- s and t consist of lowercase English letters.

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Bloomberg

19

Amazon

18

Google

2

Goldman Sachs

2

Apple

2

Yahoo

2

Spotify

2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Facebook

6

Microsoft

6

Affirm

3

Yandex

2

Qualcomm

2

Tesla

2



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

JPMorgan **5**

Oracle **4**

Cisco **4**

Snapchat **3**

Adobe **3**

Walmart Global Tech **3**

BlackRock **3**

American Express **3**

Grab **2**

IBM **2**

Visa **2**

Dell **2**

Uber

Yelp

242. Valid Anagram- Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public boolean isAnagram(String str1,String str2) {

        if(str1.length()!=str2.length()) {
            return false;
        }

        int count[] = new int[26];

        for (int i = 0; i < str1.length(); i++) {
            count[str1.charAt(i) - 97]++;
            count[str2.charAt(i) - 97]--;
        }

        for (int i = 0; i < count.length; i++) {
            if (count[i] != 0) {
                return false;
            }
        }
        return true;
    }
}
```

387. First Unique Character in a String

Description:

- ✓ Given a string str, find the first non-repeating character in it and return its index.
- ✓ If it does not exist, return -1.

Example 1:

Input: str = "leetcode"

Output: 0

Example 2:

Input: str = "loveleetcode"

Output: 2

Example 3:

Input: str = "aabb"

Output: -1

Constraints:

- $1 \leq \text{str.length} \leq 105$
- str consists of only lowercase English letters.

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon 11

Bloomberg 10

Zillow 5

Paypal 4

Uber 3

Goldman Sachs 2

Adobe 2

Oracle 2

TikTok 2

Infosys 2

Nagarro 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple 7

Microsoft 5

Qualtrics 4

Walmart Global Tech 3

Facebook 2

Yahoo 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Google 11

Visa 6

eBay 2

VMware 2

Twitter 2

ServiceNow 2

387. First Unique Character in a String - Solution

```

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public int firstUniqChar(String str) {

        int n=str.length();
        int count[] = new int[26];

        for (int i = 0; i < n; i++) {
            count[str.charAt(i)-97]++;
        }

        for (int i = 0; i < str.length(); i++) {
            if (count[str.charAt(i)-97 ] == 1) {
                return i;
            }
        }
        return -1;
    }
}

```

392. Is Subsequence

Description:

- ✓ Given two strings str and main, return true if str is a subsequence of main, or false otherwise.
- ✓ A subsequence of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not).

Example 1:

Input: s = "abc", t = "ahbgdc"

Output: true

Example 2:

Input: s = "axc", t = "ahbgdc"

Output: false

Constraints:

- $0 \leq s.length \leq 100$
- $0 \leq t.length \leq 104$
- s and t consist only of lowercase English letters.

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon **5**

Uber **3**

Google **2**

Adobe **2**

Yandex **2**



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Bloomberg 2

Apple 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Yahoo 3

Pinterest 2

Wix

392. Is Subsequence - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public boolean isSubsequence(String str,String main ) {

        int m=main.length();
        int n=str.length();

        int i=0;
        int j=0;

        while(i<m && j<n) {
            if(main.charAt(i)==str.charAt(j)) {
                j++;
            }
            i++;
        }
        return (j==n);
    }
}
```

707. Design Linked List

Description:

- ✓ Design your implementation of the linked list.
- ✓ You can choose to use a singly or doubly linked list.
- ✓ A node in a singly linked list should have two attributes: val and next.
- ✓ val is the value of the current node, and next is a pointer/reference to the next node.
- ✓ If you want to use the doubly linked list, you will need one more attribute prev to indicate the previous node in the linked list.
- ✓ Assume all nodes in the linked list are 0-indexed.

Implement the MyLinkedList class:

- a) **MyLinkedList()** :
 - ✓ Initializes the MyLinkedList object.
- b) **int get(int index)** :
 - ✓ Get the value of the indexth node in the linked list. If the index is invalid, return -1.
- c) **void addAtHead(int val)** :
 - ✓ Add a node of value val before the first element of the linked list. After the insertion, the new node will be the first node of the linked list.
- d) **void addAtTail(int val)** :
 - ✓ Append a node of value val as the last element of the linked list.
- e) **void addAtIndex(int index, int val)** :
 - ✓ Add a node of value val before the indexth node in the linked list. If index equals the length of the linked list, the node will be appended to the end of the linked list. If index is greater than the length, the node will not be inserted.
- f) **void deleteAtIndex(int index)** :
 - ✓ Delete the indexth node in the linked list, if the index is valid.

Example 1:

```
MyLinkedList myLinkedList = new MyLinkedList();  
myLinkedList.addAtHead(1);  
myLinkedList.addAtTail(3);  
myLinkedList.addAtIndex(1, 2); // linked list becomes 1->2->3  
myLinkedList.get(1);           // return 2  
myLinkedList.deleteAtIndex(1); // now the linked list is 1->3  
myLinkedList.get(1);           // return 3
```

Constraints:

- a) $0 \leq \text{index}, \text{val} \leq 1000$
- b) Please do not use the built-in LinkedList library.
- c) At most 2000 calls will be made to get, addAtHead, addAtTail, addAtIndex and deleteAtIndex.

Companies

6 months - 1 year 1 year - 2 years

Google 3 Microsoft 3 Virtu Financial 3

Companies

6 months - 1 year 1 year - 2 years

Apple 3 Amazon 2



707. Design Linked List - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class Node {
    int val;
    Node next;

    Node() {
    }

    Node(int val) {
        this.val = val;
    }
}

class MyLinkedList {

    int size;
    Node headNode;

    public MyLinkedList() {
        this.size = 0;
        this.headNode = new Node(0); // Sentinel Node
    }

    public void addAtHead(int val) {
        addAtIndex(0, val);
    }

    public void addAtTail(int val) {
        addAtIndex(size, val);
    }
}
```



```
public void addAtIndex(int index, int val) { // (0,10)

    if (index < 0 || index > size) {
        return;
    }

    Node temp = new Node(val);

    Node currNode = this.headNode;
    for(int i=0;i<index ;i++) {
        currNode = currNode.next;
    }

    temp.next = currNode.next;
    currNode.next = temp;
    size++;

}

public int get(int index) {
    if (index < 0 || index >= size) {
        return -1;
    }

    Node currNode = this.headNode;
    for(int i=0;i<index+1 ;i++) {
        currNode = currNode.next;
    }

    return currNode.val;
}
```



```
public void deleteAtIndex(int index) {  
  
    if (index < 0 || index >= size) {  
        return;  
    }  
  
    Node currNode = this.headNode;  
    for(int i=0;i<index ;i++) {  
        currNode = currNode.next;  
    }  
  
    Node temp = currNode.next;  
    currNode.next = currNode.next.next;  
    temp.next=null;  
  
    size--;  
  
}  
}
```



2. Add Two Numbers

Description:

- ✓ You are given two non-empty linked lists representing two non-negative integers.
- ✓ The digits are stored in reverse order, and each of their nodes contains a single digit.
- ✓ Add the two numbers and return the sum as a linked list.
- ✓ You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:

Input: l1 = [2,4,3], l2 = [5,6,4]

Output: [7,0,8]

Explanation: $342 + 465 = 807$.

Example 2:

Input: l1 = [0], l2 = [0]

Output: [0]

Example 3:

Input: l1 = [9,9,9,9,9,9], l2 = [9,9,9,9]

Output: [8,9,9,9,0,0,0,1]

Constraints:

- The number of nodes in each linked list is in the range [1, 100].
- $0 \leq \text{Node.val} \leq 9$
- It is guaranteed that the list represents a number that does not have leading zeros.



Java Learning Center

No.1 In Java Training & placement

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Adobe **8**

Amazon **8**

Apple **6**

Microsoft **4**

Bloomberg **3**

Expedia **3**

Facebook **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Google **8**

Uber **7**

Oracle **5**

IBM **5**

Cisco **3**

Yahoo **2**

SAP **2**

PayTM **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Capital One **4**

JPMorgan **3**

Yandex **3**

Huawei **3**

Cognizant **3**

Twitter **2**

Samsung **2**

eBay **2**

Goldman Sachs **2**

Morgan Stanley **2**

Paypal **2**

Visa **2**

ServiceNow **2**

tcs **2**

Airbnb

Wix



2. Add Two Numbers - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class Solution {

    public ListNode addTwoNumbers(ListNode list1, ListNode list2) {
        ListNode headNode = new ListNode(0);
        ListNode curr = headNode;
        int carry = 0;
        while (list1 != null || list2 != null || carry != 0) {

            int x = (list1 != null) ? list1.val : 0;
            int y = (list2 != null) ? list2.val : 0;

            int sum = carry + x + y;
            carry = sum / 10;
            curr.next = new ListNode(sum % 10);
            curr = curr.next;

            if (list1 != null)
                list1 = list1.next;

            if (list2 != null)
                list2 = list2.next;
        }

        return headNode.next;
    }
}
```

876. Middle of the Linked List

Description:

- ✓ Given the head of a singly linked list, return the middle node of the linked list.
- ✓ If there are two middle nodes, return the second middle node.

Example 1:

Input: head = [1,2,3,4,5]

Output: [3,4,5]

Explanation: The middle node of the list is node 3.

Example 2:

Input: head = [1,2,3,4,5,6]

Output: [4,5,6]

Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.

Constraints:

- The number of nodes in the list is in the range [1, 100].
- $1 \leq \text{Node.val} \leq 100$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Adobe 2

Oracle 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Amazon **5** Apple **4** Facebook **3** Qualcomm **2** Google **2**
Bloomberg **2** Uber **2**

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Microsoft **6** Samsung **2** Goldman Sachs **2** JPMorgan **2** Nvidia **2**
Walmart Global Tech

876. Middle of the Linked List - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public ListNode middleNode(ListNode headNode) {

        ListNode slow = headNode;
        ListNode fast = headNode;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
}
```

2095. Delete the Middle Node of a Linked List

Description:

- ✓ You are given the head of a linked list.
- ✓ Delete the middle node, and return the head of the modified linked list.

Example 1:

Input: head = [1,3,4,7,1,2,6]

Output: [1,3,4,1,2,6]

Example 2:

Input: head = [1,2,3,4]

Output: [1,2,4]

Example 3:

Input: head = [2,1]

Output: [2]

Constraints:

- The number of nodes in the list is in the range [1, 105].
- $1 \leq \text{Node.val} \leq 105$

Companies

6 months - 1 year

1 year - 2 years

Adobe 2



Companies

6 months - 1 year

1 year - 2 years

Microsoft 2

2095. Delete the Middle Node of a Linked List

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public ListNode deleteMiddle(ListNode headNode) {

        if(headNode.next==null) {
            return null;
        }

        ListNode slow = headNode;
        ListNode fast = headNode.next.next;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        ListNode temp = slow.next;
        slow.next = slow.next.next;
        temp.next = null;

        return headNode;
    }
}
```



203. Remove Linked List Elements

Description:

- ✓ Given the head of a linked list and an integer val, remove all the nodes of the linked list that has `Node.val == val`, and return the new head.

Example 1:

Input: head = [1,2,6,3,4,5,6], val = 6
Output: [1,2,3,4,5]

Example 2:

Input: head = [], val = 1
Output: []

Example 3:

Input: head = [7,7,7,7], val = 7
Output: []

Constraints:

- a) The number of nodes in the list is in the range [0, 104].
- b) $1 \leq \text{Node.val} \leq 50$
- c) $0 \leq \text{val} \leq 50$

Companies

6 months - 1 year

1 year - 2 years

Google 4

Adobe 3

Apple 3

Amazon 2

Bloomberg 2



Companies

6 months - 1 year

1 year - 2 years

Facebook 7

Microsoft 4

203. Remove Linked List Elements - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public ListNode removeElements(ListNode headNode, int myval) {

        ListNode dummyHead = new ListNode(0);
        dummyHead.next = headNode;

        ListNode currNode = headNode;
        ListNode prevNode = dummyHead;

        while(currNode!=null) {
            if(currNode.val==myval) {
                prevNode.next = currNode.next;
            } else {
                prevNode=currNode;
            }
            currNode = currNode.next;
        }

        return dummyHead.next;
    }
}
```

237. Delete the Node in Linked List

Description:

- ✓ There is a singly-linked list **head** and we want to delete a node **node** in it.
- ✓ You are given the node to be deleted **node**. You will not be given access to the first node of **head**.
- ✓ All the values of the linked list are unique, and it is guaranteed that the given node **node** is not the last node in the linked list.
- ✓ Delete the given node. Note that by deleting the node, we do not mean removing it from memory. We mean:
 - The value of the given node should not exist in the linked list.
 - The number of nodes in the linked list should decrease by one.
 - All the values before node should be in the same order.
 - All the values after node should be in the same order.

Example 1:

Input: head = [4,5,1,9], node = 5
Output: [4,1,9]

Example 2:

Input: head = [4,5,1,9], node = 1
Output: [4,5,9]

Constraints:

- a. The number of the nodes in the given list is in the range [2, 1000].
- b. $-1000 \leq \text{Node.val} \leq 1000$
- c. The value of each node in the list is unique.
- d. The node to be deleted is in the list and is not a tail node.

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Amazon **3**

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Bloomberg **3**

Apple **2**

Adobe **2**

Google **2**

Nagarro **2**

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Microsoft **6**

Oracle **2**

Yahoo **2**

Qualcomm **2**

Goldman Sachs **2**

Morgan Stanley **2**

237. Delete the Node in Linked List - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public void deleteNode(ListNode node) {

        ListNode nextNode = node.next;
        node.val=nextNode.val;
        node.next = nextNode.next;
        nextNode.next =null;

    }
}
```



19. Remove Nth Node From End of List

Description:

- ✓ Given the head of a linked list, remove the nth node from the end of the list and return its head.

Example 1:

Input: head = [1,2,3,4,5], n = 2
Output: [1,2,3,5]

Example 2:

Input: head = [1], n = 1
Output: []

Example 3:

Input: head = [1,2], n = 1
Output: [1]

Constraints:

- a) The number of nodes in the list is sz.
- b) $1 \leq sz \leq 30$
- c) $0 \leq \text{Node.val} \leq 100$
- d) $1 \leq n \leq sz$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon 3

Apple 3

Adobe 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Facebook **16** Google **4** Microsoft **4** Bloomberg **4** Yahoo **2**
 eBay **2** Nvidia **2** VMware **2** American Express **2** Intel **2**
 Dell **2**

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Uber **3** Walmart Global Tech **3** Oracle **2** Yandex **2** Salesforce **2**
 Paypal **2** Nagarro **2**

19. Remove Nth Node From End of List - Solution

```

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class Solution {
    public ListNode removeNthFromEnd(ListNode headNode, int n) {

        ListNode currNode = headNode;

        for(int i=1;i<=n && currNode!=null;i++) {
            currNode = currNode.next;
        }
    }
}

```



```
        if(currNode==null) {  
            return headNode.next;  
        }  
  
        ListNode prevNode = headNode;  
        while(currNode.next!=null) {  
            prevNode = prevNode.next;  
            currNode = currNode.next;  
        }  
  
        prevNode.next = prevNode.next.next;  
  
        return headNode;  
    }  
}
```

206. Reverse LinkedList

Description:

- ✓ Given the head of a singly linked list, reverse the list, and return the reversed list.

Example 1:

Input : 1 -> 2 -> 3 -> 4 -> 5

Output : 5 -> 4 -> 3 -> 2 -> 1

Example 2:

Input : 1 -> 2

Output : 2 -> 1

Example 3:

Input : []

Output : []

Constraints:

- The number of nodes in the list is the range [0, 5000].
- $-5000 \leq \text{Node.val} \leq 5000$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple **5**

Amazon **3**

Microsoft **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Bloomberg **6**

Adobe **5**

Oracle **4**

Facebook **3**

Google **3**

ServiceNow **3**

Uber **2**

VMware **2**

Nagarro **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Nvidia 6

Yandex 6

Paypal 3

Intuit 3

eBay 3

IBM 3

Cisco 3

Yahoo 2

Samsung 2

Dell 2

PayTM 2

JPMorgan 2

HBO 2

TikTok 2

Canonical 2

Snapchat

Zenefits

Yelp

Twitter

206. Reverse LinkedList- Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public ListNode reverseList(ListNode headNode) {

        ListNode prevNode = null;
        ListNode nextNode = null;

        ListNode currNode = headNode;

        while(currNode!=null) {
            nextNode = currNode.next;
            currNode.next=prevNode;

            prevNode=currNode;
            currNode=nextNode;
        }
        return prevNode;
    }
}
```

234. Palindrome Linked List

Description:

- ✓ Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

Example 1:

Input: head = [1,2,2,1]

Output: true

Example 2:

Input: head = [1,2]

Output: false

Constraints:

- a) The number of nodes in the list is in the range [1, 105].
- b) $0 \leq \text{Node.val} \leq 9$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon **5**

Microsoft **3**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple **7**

Facebook **5**

Google **5**

Adobe **4**

Bloomberg **3**

Yahoo **3**

Intuit **3**

Spotify **3**

ServiceNow **2**

Accenture **2**



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

VMware 2

Oracle 2

Paypal 2

IXL

234. Palindrome Linked List

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {

    public ListNode middleNode(ListNode headNode) {

        ListNode slow = headNode;
        ListNode fast = headNode;

        while (fast.next != null && fast.next.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        return slow;
    }

    public ListNode reverseList(ListNode headNode) {

        ListNode prevNode = null;
        ListNode nextNode = null;

        ListNode currNode = headNode;

        while (currNode != null) {
            nextNode = currNode.next;
            currNode.next = prevNode;
        }
    }
}
```



```
        prevNode = currNode;
        currNode = nextNode;

    }
    return prevNode;
}

public boolean isPalindrome(ListNode headNode) {
    if (headNode == null) {
        return true;
    }

    ListNode middle = middleNode(headNode);

    ListNode afterMiddle = reverseList(middle.next);

    ListNode part1 = headNode;
    ListNode part2 = afterMiddle;
    boolean flag = true;
    while (flag && part2 != null) {
        if (part1.val != part2.val) {
            flag = false;
        }
        part1 = part1.next;
        part2 = part2.next;
    }
    return flag;
}
}
```

328. Odd Even Linked List

Description:

- ✓ Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return the reordered list.
- ✓ The first node is considered odd, and the second node is even, and so on.
- ✓ Note that the relative order inside both the even and odd groups should remain as it was in the input.
- ✓ You must solve the problem in $O(1)$ extra space complexity and $O(n)$ time complexity.

Example 1:

Input: head = [1,2,3,4,5] Output: [1,3,5,2,4]

Example 2:

Input: head = [2,1,3,5,6,4,7] Output: [2,3,6,7,1,5,4]

Constraints:

- The number of nodes in the linked list is in the range [0, 104].
- $-10^6 \leq \text{Node.val} \leq 10^6$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Google 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Amazon **6** Apple **4** Yahoo **2** Facebook **2**

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Microsoft **5** Bloomberg **4** Adobe **4** eBay **2** VMware **2**
Oracle **2** TikTok **2**

328. Odd Even Linked List

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public ListNode oddEvenList(ListNode headNode) {
        if (headNode == null)
            return null;
        ListNode odd = headNode;
        ListNode even = headNode.next;
        ListNode evenHead = even;

        while (even != null && even.next != null) {
            odd.next = even.next;
            odd = odd.next;
            even.next = odd.next;
            even = even.next;
        }
        odd.next = evenHead;
        return headNode;
    }
}
```

141. Linked List Cycle

Description:

- ✓ Given head, the head of a linked list, determine if the linked list has a cycle in it.
- ✓ There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter.
- ✓ Return true if there is a cycle in the linked list. Otherwise, return false.

Example 1:

Input : 3 -> 2 -> 0 -> 4 -> 2

Output : true

Example 2:

Input : 1 -> 2 -> 1

Output : true

Constraints:

- The number of the nodes in the list is in the range $[0, 10^4]$.
- $-10^5 \leq \text{Node.val} \leq 10^5$
- pos is -1 or a valid index in the linked-list.

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Apple 7 Adobe 4 Bloomberg 3 Oracle 3 Cisco 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Microsoft 13 Spotify 6 Google 5 Visa 5 Goldman Sachs 4

Nvidia 4 Facebook 3 Yahoo 3 Salesforce 3 Intel 2 Sprinklr 2

Splunk 2

141. Linked List Cycle- Solution

```

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public boolean hasCycle(ListNode headNode) {
        if (headNode == null) {
            return false;
        }
        ListNode slow = headNode;
        ListNode fast = headNode;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) {
                return true;
            }
        }
        return false;
    }
}

```

160. Intersection of Two Linked Lists

Description:

- ✓ Given the heads of two singly linked-lists **headA** and **headB**, return the node at which the two lists intersect. If the two linked lists have no intersection at all, return null.

Example 1:

Input :

headA : 4 -> 1 -> 8 -> 4 -> 5

HeadB : 5 -> 6 -> 1 -> 8 -> 4 -> 5

Output: Intersected at '8'

Input :

headA : 2 -> 6 -> 4

HeadB : 1 -> 5

Output: No intersection

Constraints:

- a) The number of nodes of listA is in the m.
- b) The number of nodes of listB is in the n.

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Airbnb 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Apple 6 Microsoft 5 Amazon 2 Bloomberg 2 Facebook 2

Oracle 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

LinkedIn 10 Adobe 6 TikTok 6 Nvidia 5 Google 2

Goldman Sachs 2 Samsung 2 Spotify 2 Nutanix 2

160. Intersection of 2 Linked Lists

```

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public ListNode getIntersectionNode(ListNode head1, ListNode head2) {
        ListNode currNode1 = head1;
        ListNode currNode2 = head2;
        while (currNode1 != currNode2) {
            currNode1 = (currNode1 == null) ? head2 : currNode1.next;
            currNode2 = (currNode2 == null) ? head1 : currNode2.next;
        }
        return currNode1;
    }
}

```

21. Merge Two Sorted Lists

Description:

- ✓ You are given the heads of two sorted linked lists **list1** and **list2**.
- ✓ Merge the two lists in a one sorted list. The list should be made by splicing together the nodes of the first two lists.
- ✓ Return the head of the merged linked list.

Example 1:

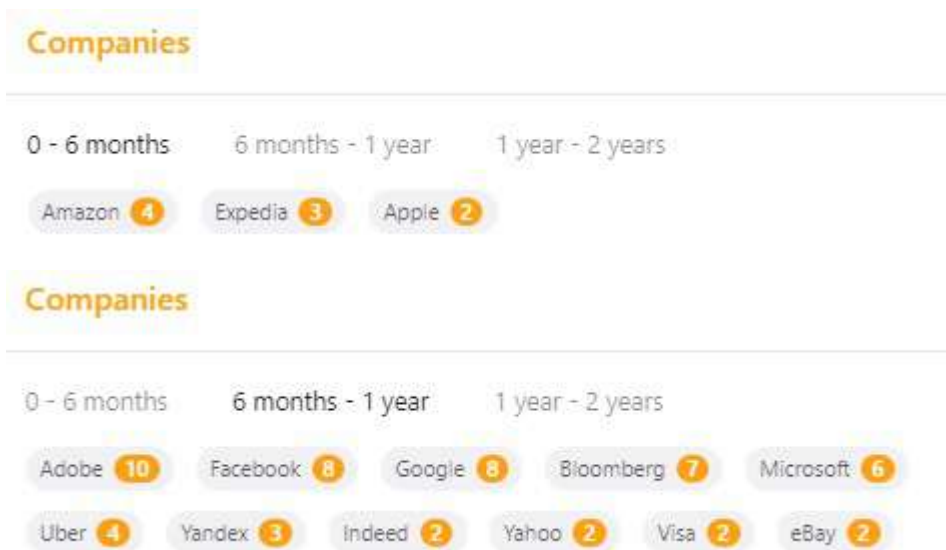
Input: list1 = [1,2,4], list2 = [1,3,4] **Output:** [1,1,2,3,4,4]

Example 2:

Input: list1 = [], list2 = [] **Output:** []

Constraints:

- The number of nodes in both lists is in the range [0, 50].
- $-100 \leq \text{Node.val} \leq 100$
- Both list1 and list2 are sorted in non-decreasing order.





Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

VMware 4

Oracle 4

ByteDance 4

Shopee 4

LinkedIn 2

Accenture 2

Intuit 2

Walmart Global Tech 2

GoDaddy 2

Roblox 2

Wix

21. Merge Two Sorted Lists

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {

        ListNode head = new ListNode(-1);
        ListNode tail = head;

        while (list1 != null && list2 != null) {
            if (list1.val <= list2.val) {
                tail.next = list1;
                list1 = list1.next;
            } else {
                tail.next = list2;
                list2 = list2.next;
            }
            tail = tail.next;
        }
        tail.next = (list1 == null) ? list2 : list1;
        return head.next;
    }
}
```



20. Valid Parentheses

Description:

- ✓ Given a string `s` containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.
- ✓ An input string is valid if:
 - Open brackets must be closed by the same type of brackets.
 - Open brackets must be closed in the correct order.
 - Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: `str = "()"`

Output: `true`

Example 2:

Input: `str = "(){}"`

Output: `true`

Example 3:

Input: `str = "["`

Output: `false`

Constraints:

- a) $1 \leq s.length \leq 104$
- b) `s` consists of parentheses only '`(){}[]`'.



Java Learning Center

No.1 In Java Training & placement

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon **9**

Bloomberg **5**

Google **5**

Apple **3**

Microsoft **2**

Adobe **2**

Yahoo **2**

JPMorgan **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

BlackRock **18**

Facebook **16**

Spotify **9**

LinkedIn **8**

Uber **6**

Zillow **6**

Yandex **5**

Oracle **5**

Cisco **3**

IBM **3**

Netflix **3**

tcs **3**

Mathworks **2**

eBay **2**

VMware **2**

Qualcomm **2**

Lyft **2**

Goldman Sachs **2**

Walmart Global Tech **2**

Intel **2**

SAP **2**

Twitch **2**

TikTok **2**

Société Générale **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Expedia **16**

Bolt **8**

ServiceNow **6**

Salesforce **6**

Paypal **4**

Morgan Stanley **4**

Atlassian **3**

Booking.com **3**

Nvidia **3**

Splunk **3**

Arista Networks **3**

Twilio **2**

Tesla **2**

Barclays **2**

Twitter **2**

Siemens **2**

Visa **2**

Huawei **2**

Intuit **2**

Dataminr **2**

Infosys **2**

Accolite **2**

Nagarro **2**

MakeMyTrip **2**

Airbnb

Zenefits

20. Valid Parentheses - Solution

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
**/
```



```
class Solution {

    private Map<Character, Character> map ;

    public Solution() {
        this.map = new HashMap<Character, Character>();
        this.map.put('(', ')');
        this.map.put('{', '}');
        this.map.put('[', ']');
    }

    public boolean isValid(String str) {

        Deque<Character> mystack = new ArrayDeque<>();

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);

            if (map.containsKey(ch)) {
                mystack.push(ch);
            } else {
                if(mystack.isEmpty()) {
                    return false;
                }
                char open = mystack.pop();
                if ( map.get(open)!=ch) {
                    return false;
                }
            }
        }
        } // for end

        return mystack.isEmpty();
    }
}
```

155. Min Stack

Description:

- ✓ Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.
- ✓ **Implement the MinStack class:**
 - a) **MinStack()**: initializes the stack object.
 - b) **void push(int val)**: pushes the element val onto the stack.
 - c) **void pop()**: removes the element on the top of the stack.
 - d) **int top()**: gets the top element of the stack.
 - e) **int getMin()**: retrieves the minimum element in the stack.
- ✓ You must implement a solution with $O(1)$ time complexity for each function.

Constraints:

- a) $-2^{31} \leq \text{val} \leq 2^{31} - 1$
- b) Methods pop, top and getMin operations will always be called on non-empty stacks.
- c) At most $3 * 10^4$ calls will be made to push, pop, top, and getMin.

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Expedia 3 Amazon 2 Bloomberg 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Microsoft 6 Apple 3 Adobe 2



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Facebook

9

VMware

5

Google

4

Salesforce

4

Oracle

4

Walmart Global Tech

3

Goldman Sachs

3

Yahoo

2

Airtel

2

Arcesium

2

Uber

Snapchat

Zenefits

155. Min Stack - Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class MinStack {
    Deque<int[]> mystack = new ArrayDeque<>();

    public void push(int element) {
        if (mystack.isEmpty()) {
            mystack.push(new int[]{element,element});
            return;
        }
        int currMin = mystack.peek()[1];
        mystack.push(new int[]{element, Math.min(element, currMin)});
    }
    public void pop() {
        mystack.pop();
    }
    public int top() {
        return mystack.peek()[0];
    }
    public int getMin() {
        return mystack.peek()[1];
    }
}
```



1472. Design Browser History

Description:

- ✓ You have a browser of one tab where you start on the homepage and you can visit another url, get back in the history number of steps or move forward in the history number of steps.

- ✓ **Implement the BrowserHistory class:**

a) BrowserHistory(string homepage) :

Initializes the object with the homepage of the browser.

b) void visit(string url) :

Visits url from the current page. It clears up all the forward history.

c) string back(int steps) :

Move steps back in history. If you can only return x steps in the history and steps > x, you will return only x steps. Return the current url after moving back in history at most steps.

d) string forward(int steps) :

Move steps forward in history. If you can only forward x steps in the history and steps > x, you will forward only x steps. Return the current url after forwarding in history at most steps.

Constraints:

- a) $1 \leq \text{homepage.length} \leq 20$
- b) $1 \leq \text{url.length} \leq 20$
- c) $1 \leq \text{steps} \leq 100$
- d) homepage and url consist of '.' or lower case English letters.
- e) At most 5000 calls will be made to visit, back, and forward.

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Bloomberg 4

Roblox 4

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Amazon 6

DoorDash 2

Oracle 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Google 4

Microsoft 2

1472. Design Browser History

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class BrowserHistory {

    Deque<String> backHistory, forwardHistory;
    String current;

    public BrowserHistory(String homepage) {
        backHistory = new ArrayDeque<>();
        forwardHistory = new ArrayDeque<String>();
        current = homepage;
    }
}
```



```
public void visit(String url) {
    backHistory.push(current);
    current = url;
    forwardHistory = new ArrayDeque<String>();
}

public String back(int steps) {
    while(steps > 0 && !backHistory.isEmpty()) {
        forwardHistory.push(current);
        current = backHistory.pop();
        steps--;
    }
    return current;
}

public String forward(int steps) {
    while(steps > 0 && !forwardHistory.isEmpty()) {
        backHistory.push(current);
        current = forwardHistory.pop();
        steps--;
    }
    return current;
}
}
```



71. Simplify Path

Description:

- ✓ Given a string path, which is an absolute path (starting with a slash '/') to a file or directory in a Unix-style file system, convert it to the simplified canonical path.
- ✓ In a Unix-style file system, a period '.' refers to the current directory, a double period '..' refers to the directory up a level, and any multiple consecutive slashes (i.e. '//') are treated as a single slash '/'. For this problem, any other format of periods such as '...' are treated as file/directory names.
- ✓ The canonical path should have the following format:
 - The path starts with a single slash '/'.
 - Any two directories are separated by a single slash '/'.
 - The path does not end with a trailing '/'.
 - The path only contains the directories on the path from the root directory to the target file or directory (i.e., no period '.' or double period '..')
- ✓ Return the simplified canonical path.

Example 1:

Input: path = "/home/" **Output:** "/home"

Example 2:

Input: path = "/../" **Output:** "/"

Example 3:

Input: path = "/home//foo/" **Output:** "/home/foo"

Constraints:

- a) $1 \leq \text{path.length} \leq 3000$
- b) path consists of English letters, digits, period '.', slash '/' or '_'.
- c) path is a valid absolute Unix path.

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Facebook 2 Yandex 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Amazon 5 Microsoft 3 Google 3 Uber 2 Oracle 2 tcs 2

Companies

0 - 6 months 6 months - 1 year 1 year - 2 years

Apple 3 Wish 3 Bloomberg 2 VMware 2 Pinterest 2



71. Simplify Path- Solution

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public String simplifyPath(String path) {

        Stack<String> mystack = new Stack<>();
        String[] directoryList = path.split("/");

        for (String directory : directoryList) {
            if (directory.equals(".") || directory.isEmpty()) {
                continue;
            } else if (directory.equals("..")) {
                if (!mystack.isEmpty()) {
                    mystack.pop();
                }
            } else {
                mystack.add(directory);
            }
        }

        StringBuilder result = new StringBuilder();
        for (String dir : mystack) {
            result.append("/");
            result.append(dir);
        }

        return result.length() > 0 ? result.toString() : "/" ;
    }
}
```



394. Decode String

Description:

- ✓ Given an encoded string, return its decoded string.
- ✓ The encoding rule is: k[encoded_string], where the encoded_string inside the square brackets is being repeated exactly k times. Note that k is guaranteed to be a positive integer.
- ✓ You may assume that the input string is always valid;

Example 1:

Input: s = "3[a]2[bc]"

Output: "aaabcbc"

Example 2:

Input: s = "3[a2[c]]"

Output: "accaccacc"

Example 3:

Input: s = "2[abc]3[cd]ef"

Output: "abcbcccdcdcdcf"

Constraints:

- 1 <= s.length <= 30
- s consists of lowercase English letters, digits, and square brackets '['].
- s is guaranteed to be a valid input.
- All the integers in s are in the range [1, 300].



Java Learning Center

No.1 In Java Training & placement

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Bloomberg 14

Google 4

TikTok 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon 14

Microsoft 9

Apple 6

Adobe 6

Uber 6

C3 IoT 6

ByteDance 4

Yahoo 3

Salesforce 2

eBay 2

Qualtrics 2

Square 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Cisco 17

Snapchat 15

Facebook 13

Oracle 9

Intuit 4

Sumologic 4

Cruise Automation 3

Walmart Global Tech 3

HBO 3

SAP 2

Goldman Sachs 2

ServiceNow 2

Jump Trading 2

Millennium 2

Coupang

Yelp

394. Decode String - Solution

```
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
  
class Solution {
```



```
String decodeString(String str) {

    Stack<Integer> digitStack = new Stack<>();
    Stack<StringBuilder> resultStack = new Stack<>();

    StringBuilder resultString = new StringBuilder();
    int k = 0;

    for (char ch : str.toCharArray()) {

        if (Character.isDigit(ch)) {
            k = k * 10 + ch - '0';
        } else if (ch == '[') {
            digitStack.push(k);
            resultStack.push(resultString);
            resultString = new StringBuilder();
            k = 0;
        } else if (ch == ']') {

            StringBuilder decodedString = resultStack.pop();
            int digit = digitStack.pop();
            for (int i=1;i<=digit;i++) {
                decodedString.append(resultString);
            }

            resultString = decodedString;

        } else {
            resultString.append(ch);
        }
    }
    return resultString.toString();
}
```



622. Design Circular Queue

Description:

- ✓ **Design your implementation of the circular queue.**
- ✓ The circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle, and the last position is connected back to the first position to make a circle. It is also called "Ring Buffer".
- ✓ **Implement the MyCircularQueue class:**
 - 1) **MyCircularQueue(k)**: Initializes the object with the size of the queue to be k.
 - 2) **int Front()**: Gets the front item from the queue. If the queue is empty, return -1.
 - 3) **int Rear()**: Gets the last item from the queue. If the queue is empty, return -1.
 - 4) **boolean enQueue(int value)**: Inserts an element into the circular queue. Return true if the operation is successful.
 - 5) **boolean deQueue()**: Deletes an element from the circular queue. Return true if the operation is successful.
 - 6) **boolean isEmpty()**: Checks whether the circular queue is empty or not.
 - 7) **boolean isFull()**: Checks whether the circular queue is full or not.

You must solve the problem without using the built-in queue data structure in your programming language.

Constraints:

- a) $1 \leq k \leq 1000$
- b) $0 \leq \text{value} \leq 1000$
- c) At most 3000 calls will be made to enQueue, deQueue, Front, Rear, isEmpty, and isFull.



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple 3

Amazon 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Bloomberg 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Microsoft 13

Facebook 8

Nvidia 6

Airbnb 4

Intuit 4

Rubrik 3

Google 2

622. Design Circular Queue

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
* */
```

```
class MyCircularQueue {
```

```
    private int capacity; // size of array
```

```
    private int size; // size of the Queue
```

```
    private int front;
```

```
    private Integer myarray[];
```



```
public MyCircularQueue(int capacity) {
    this.capacity = capacity;
    this.size = 0;
    this.front = 0;
    this.myarray = new Integer[capacity];
}

public boolean enqueue(int element) {
    if (size == capacity) {
        return false;
    }

    myarray[(front + size) % capacity] = element;
    size++;
    return true;
}

public boolean dequeue() {
    if (size == 0) {
        return false;
    }
    myarray[front] = null;
    front = (front + 1) % capacity;
    size--;
    return true;
}

public int Front() {
    if (size == 0)
        return -1;

    return myarray[front];
}
```



```
public int Rear() {  
    if (size == 0)  
        return -1;  
    int rear = (front + size - 1) % capacity;  
    return myarray[rear];  
}  
  
public int size() {  
    return size;  
}  
  
public boolean isEmpty() {  
    return (size == 0);  
}  
  
public boolean isFull() {  
    return (size == capacity);  
}  
}
```



641. Design Circular Deque

Description:

- ✓ Design your implementation of the circular double-ended queue (deque).
- ✓ Implement the MyCircularDeque class:
 - 1) **MyCircularDeque(int k)**: Initializes the deque with a maximum size of k.
 - 2) **boolean insertFront()**: Adds an item at the front of Deque. Returns true if the operation is successful, or false otherwise.
 - 3) **boolean insertLast()**: Adds an item at the rear of Deque. Returns true if the operation is successful, or false otherwise.
 - 4) **boolean deleteFront()**: Deletes an item from the front of Deque. Returns true if the operation is successful, or false otherwise.
 - 5) **boolean deleteLast()**: Deletes an item from the rear of Deque. Returns true if the operation is successful, or false otherwise.
 - 6) **int getFront()**: Returns the front item from the Deque. Returns -1 if the deque is empty.
 - 7) **int getRear()**: Returns the last item from Deque. Returns -1 if the deque is empty.
 - 8) **boolean isEmpty()**: Returns true if the deque is empty, or false otherwise.
 - 9) **boolean isFull()**: Returns true if the deque is full, or false otherwise.

Constraints:

- a) $1 \leq k \leq 1000$
- b) $0 \leq \text{value} \leq 1000$
- c) At most 2000 calls will be made to insertFront, insertLast, deleteFront, deleteLast, getFront, getRear, isEmpty, isFull.



Companies

1 year - 2 years

Goldman Sachs **2**

641. Design Circular Deque

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class MyCircularDeque {
    private int front;
    private int rear;
    private int size;
    private int capacity;
    private Integer myarray[];

    public MyCircularDeque(int capacity) {
        this.capacity = capacity;
        myarray = new Integer[capacity];
        this.front = -1;
        this.rear = 0;
        this.size = 0;
    }

    public boolean isFull() {
        return (size == capacity);
    }

    public int size() {
        return size;
    }
}
```



```
public boolean isEmpty() {
    return (size == 0);
}

public int getFront() {
    if (isEmpty()) {
        return -1;
    }
    return myarray[front];
}

public int getRear() {
    if (isEmpty()) {
        return -1;
    }
    return myarray[rear];
}

public boolean insertFront(int element) {
    if (isFull()) {
        return false;
    }

    if (front == -1) {
        front = 0;
        rear = 0;
    } else if (front == 0) {
        front = capacity - 1;
    } else {
        front = front - 1;
    }

    myarray[front] = element;
    size++;
    return true;
}
```



```
public boolean insertLast(int element) {
    if (isFull())
        return false;

    if (front == -1) {
        front = 0;
        rear = 0;
    } else if (rear == capacity-1) {
        rear = 0;
    } else {
        rear = rear + 1;
    }

    myarray[rear] = element;
    size++;
    return true;
}

public boolean deleteFront() {
    if (isEmpty())
        return false;

    myarray[front] = null;
    size--;

    if (front == rear) {
        front = -1;
        rear = -1;
    } else if (front == capacity-1) {
        front = 0 ;
    } else {
        front = front+1;
    }
    return true;
}
```



```
public boolean deleteLast() {  
    if (isEmpty()) {  
        return false;  
    }  
  
    myarray[rear] = null;  
    size--;  
  
    if (front == rear) {  
        front = -1;  
        rear = -1;  
    } else if (rear == 0) {  
        rear = capacity - 1;  
    } else {  
        rear = rear - 1;  
    }  
  
    return true;  
}  
}
```

225. Implement Stack using Queues

Description:

- ✓ Implement a last-in-first-out (LIFO) stack using only two queues.
- ✓ The implemented stack should support all the functions of a normal stack (push, top, pop, and empty).

Implement the MyStack class:

- 1) **void push(int x):** Pushes element x to the top of the stack.
- 2) **int pop():** Removes the element on the top of the stack and returns it.
- 3) **int top():** Returns the element on the top of the stack.
- 4) **boolean empty():** Returns true if the stack is empty, false otherwise.

Notes:

- ✓ You must use only standard operations of a queue, which means that only push to back, peek/pop from front, size and is empty operations are valid.

Constraints:

- a) $1 \leq x \leq 9$
- b) At most 100 calls will be made to push, pop, top, and empty.
- c) All the calls to pop and top are valid.

Companies

6 months - 1 year

1 year - 2 years

Amazon 2

Google 2



Companies

6 months - 1 year

1 year - 2 years

Bloomberg 4

PayPal 3

Microsoft 2

225. Implement Stack using Queues

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class MyStack {

    Queue<Integer> myque1;
    Queue<Integer> myque2;

    public MyStack() {
        myque1 = new LinkedList<>();
        myque2 = new LinkedList<>();
    }

    public void push(int element) {

        //Step1: Add Element to myque2

        myque2.offer(element);

        //Step2: Move Elements from myque1 to myque2

        while(!myque1.isEmpty()) {
            myque2.offer(myque1.poll());
        }
    }
}
```



//Step3: Swap

```
        Queue<Integer> temp = myque1;
        myque1= myque2;
        myque2=temp;

    }

    public int pop() {
        return myque1.poll();
    }

    public int top() {
        return myque1.peek();
    }

    public boolean empty() {
        return myque1.isEmpty();
    }
}
```

232. Implement Queue using Stacks

Description:

- ✓ Implement a first in first out (FIFO) queue using only two stacks.
- ✓ The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).

Implement the MyQueue class:

- 1) **void push(int x):** Pushes element x to the back of the queue.
- 2) **int pop():** Removes the element from the front of the queue and returns that element
- 3) **int peek():** Returns the element at the front of the queue.
- 4) **boolean empty():** Returns true if the queue is empty, false otherwise.

Notes:

- ✓ You must use only standard operations of a stack, which means only push to top, peek/pop from top, size, and is empty operations are valid.

Constraints:

- a) $1 \leq x \leq 9$
- b) At most 100 calls will be made to push, pop, peek, and empty.
- c) All the calls to pop and peek are valid.

Companies

6 months - 1 year

1 year - 2 years

Amazon 3

Microsoft 2

Apple 2



Companies

6 months - 1 year

1 year - 2 years

Goldman Sachs 5

VMware 3

eBay 2

ByteDance 2

Docusign 2

Bloomberg 2

Google 2

Oracle 2

232. Implement Queue using Stacks

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class MyQueue {

    Stack<Integer> mystack1;
    Stack<Integer> mystack2;

    public MyQueue() {
        mystack1 = new Stack<>();
        mystack2 = new Stack<>();
    }

    public void push(int element) {
        mystack1.push(element);
    }

    public int pop() {

        if (mystack2.isEmpty()) {
            while (!mystack1.isEmpty()) {
                mystack2.push(mystack1.pop());
            }
        }
    }
}
```



```
        int popElement = mystack2.pop();

        while (!mystack2.isEmpty()) {
            mystack1.push(mystack2.pop());
        }

        return popElement;
    }

    public int peek() {
        if (mystack2.isEmpty()) {
            while (!mystack1.isEmpty()) {
                mystack2.push(mystack1.pop());
            }
        }
        int topElement = mystack2.peek();

        while (!mystack2.isEmpty()) {
            mystack1.push(mystack2.pop());
        }

        return topElement;
    }

    public boolean empty() {
        return mystack1.isEmpty();
    }
}
```



239. Sliding Window Maximum

Description:

- ✓ You are given an array of integers nums, there is a sliding window of size k which is moving from the very left of the array to the very right.
- ✓ You can only see the k numbers in the window.
- ✓ Each time the sliding window moves right by one position.
- ✓ Return the max sliding window.

Example 1:

Input: nums = [1,3,-1,-3,5,3,6,7], k = 3

Output: [3,3,5,5,6,7]

Example 2:

Input: nums = [1], k = 1

Output: [1]

Constraints:

- a) $1 \leq \text{nums.length} \leq 105$
- b) $-104 \leq \text{nums}[i] \leq 104$
- c) $1 \leq k \leq \text{nums.length}$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon 4

Yandex 3

Uber 2

Microsoft 2

Google 2

Adobe 2



Java Learning Center

No.1 In Java Training & placement

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple **5**

DoorDash **4**

Citadel **3**

Facebook **2**

Bloomberg **2**

HRT **2**

TikTok **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Goldman Sachs **9**

Salesforce **9**

Coinbase **8**

ByteDance **6**

Twitter **5**

Twilio **5**

VMware **4**

Booking.com **4**

DE Shaw **4**

Rubrik **3**

Walmart Labs **2**

Akuna Capital **2**

Atlassian **2**

TuSimple **2**

Quora **2**

Cruise Automation **2**

Capgemini **2**

Zenefits

239. Sliding Window Maximum

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {

        Deque<Integer> mydeque = new ArrayDeque<>();

        int n = nums.length;
        int resultArr[] = new int[n - k + 1];

        if (n * k == 0) {
            return new int[0];
        }
    }
}
```



```
if (k == 1) {
    return nums;
}

for(int i=0;i<k;i++) {

    while(!mydeque.isEmpty() && nums[i]>=nums[mydeque.peekLast()]) {
        mydeque.removeLast();
    }

    mydeque.addLast(i);
}

resultArr[0] = nums[mydeque.peek()];

for(int i=k;i<n;i++) {

    if(!mydeque.isEmpty() && mydeque.peek()==i-k) {
        mydeque.removeFirst();
    }

    while(!mydeque.isEmpty() && nums[i]>=nums[mydeque.peekLast()]) {
        mydeque.removeLast();
    }

    mydeque.addLast(i);
    resultArr[i-k+1] = nums[mydeque.peek()];
}

return resultArr;
}
}
```



704. Binary Search

Description:

- ✓ Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`.
- ✓ If `target` exists, then return its index. Otherwise, return `-1`.
- ✓ You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [-1,0,3,5,9,12]`, `target = 9`

Output: `4`

Example 2:

Input: `nums = [-1,0,3,5,9,12]`, `target = 2`

Output: `-1`

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 < \text{nums}[i], \text{target} < 10^4$
- All the integers in `nums` are unique.
- `nums` is sorted in ascending order.

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple 3

Amazon 2

Adobe 2



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Google **7**

Uber **4**

Facebook **2**

Yahoo **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Microsoft **8**

Bloomberg **5**

SAP **3**

Yandex **3**

tcs **3**

Goldman Sachs **2**

Infosys **2**

704. Binary Search

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public int search(int[] arr, int element) {
        int n = arr.length;
        int low = 0;    int high = n - 1;

        while (low <= high) {
            int mid = (low + high) / 2;
            if (element == arr[mid] ) {
                return mid;
            }
            else if (element > arr[mid]) {
                low = mid + 1;
            }
            else if (element < arr[mid]) {
                high = mid - 1;
            }
        }
    }
}
```

```
        return -1;  
    }  
}
```

34. Find First and Last Position of Element in Sorted Array

Description:

- ✓ Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given target value.
- ✓ If target is not found in the array, return `[-1, -1]`.
- ✓ You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [5,7,7,8,8,10]`, `target = 8`
Output: `[3,4]`

Example 2:

Input: `nums = [5,7,7,8,8,10]`, `target = 6`
Output: `[-1,-1]`

Constraints:

- $0 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$
- `nums` is a non-decreasing array.
- $-10^9 \leq \text{target} \leq 10^9$



Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Adobe **4**

Apple **2**

Facebook **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon **14**

Bloomberg **5**

Intuit **4**

Google **3**

Uber **2**

Samsung **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Microsoft **17**

LinkedIn **16**

Qualtrics **4**

ByteDance **4**

instacart **3**

Yahoo **2**

Nutanix **2**

Oracle **2**

Flipkart **2**

Splunk **2**

TikTok **2**

SAP **2**

34. Find First and Last Position of Element in Sorted Array

```
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
class Solution {  
    private static int firstOccurrence(int arr[], int element) {  
  
        int n = arr.length;  
        int low = 0;  
        int high = n - 1;
```



```
while (low <= high) {
    int mid = (low + high) / 2;
    if (element > arr[mid]) {
        low = mid + 1;
    } else if (element < arr[mid]) {
        high = mid - 1;
    } else {
        if (mid == 0 || arr[mid - 1] != arr[mid])
            return mid;
        else
            high = mid - 1;
    }
}
return -1;
}
```

```
private static int lastOccurrence(int arr[], int element) {

    int n = arr.length;
    int low = 0;
    int high = n - 1;

    while (low <= high) {
        int mid = (low + high) / 2;
        if (element > arr[mid]) {
            low = mid + 1;
        } else if (element < arr[mid]) {
            high = mid - 1;
        } else {
            if (mid == n-1 || arr[mid] != arr[mid+1])
                return mid;
            else
                low = mid + 1;
        }
    }
    return -1;
}
```



```
public int[] searchRange(int[] arr, int element) {  
    int first = firstOccurrence(arr, element);  
  
    if(first==-1) {  
        return new int[]{-1, -1};  
    }  
  
    int last = lastOccurrence(arr,element);  
    return new int[]{first, last};  
}
```

69. Sqrt(num)

Description:

- ✓ Given a non-negative integer x, return the square root of x rounded down to the nearest integer. The returned integer should be non-negative as well.
- ✓ You must not use any built-in exponent function or operator.

Example 1:

Input: x = 4 Output: 2

Example 2:

Input: x = 8 Output: 2

Constraints:

a) $0 \leq x \leq 2^{31} - 1$



Java Learning Center

No.1 In Java Training & placement

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon **6**

Apple **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Bloomberg **5**

Microsoft **3**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

LinkedIn **15**

Google **7**

Adobe **7**

Uber **5**

Facebook **3**

TikTok **2**



69. Sqrt(num)

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class Solution {
    public int mySqrt(int num) {

        if (num < 2)
            return num;

        int low = 2;
        int high = num/2;
        long result=0;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            result = (long)mid * mid;

            if (result == num) {
                return mid;
            }
            else if (result > num) {
                high = mid - 1;
            }
            else if (result < num) {
                low = mid + 1;
            }
        }

        return high;
    }
}
```



33. Search in Rotated Sorted Array

Description:

- ✓ There is an integer array `nums` sorted in ascending order (with distinct values).
- ✓ Prior to being passed to your function, `nums` is possibly rotated at an unknown pivot index `k` ($1 \leq k < \text{nums.length}$)
- ✓ For example, `[0,1,2,4,5,6,7]` might be rotated at pivot index 3 and become `[4,5,6,7,0,1,2]`.
- ✓ Given the array `nums` after the possible rotation and an integer `target`, return the index of `target` if it is in `nums`, or `-1` if it is not in `nums`.
- ✓ You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0` Output: 4

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 3` Output: -1

Constraints:

- a) $1 \leq \text{nums.length} \leq 5000$
- b) $-104 \leq \text{nums}[i] \leq 104$
- c) All values of `nums` are unique.
- d) `nums` is an ascending array that is possibly rotated.
- e) $-104 \leq \text{target} \leq 104$

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Apple 5

Amazon 3

Google 3

Uber 3

Yahoo 2

Oracle 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Bloomberg 11

Adobe 6

Microsoft 5

Media.net 5

TikTok 3

IXL 2

Facebook 2

PayPal 2

Visa 2

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

LinkedIn 40

ByteDance 10

VMware 6

Walmart Labs 6

Goldman Sachs 6

ServiceNow 6

Expedia 5

Flipkart 3

Salesforce 3

Morgan Stanley 3

Splunk 3

Nvidia 2

Airbnb 2

Intuit 2

Paytm 2

Qualtrics 2

MakeMyTrip 2

Airtel 2

33. Search in Rotated Sorted Array

/*

* @Author : Srinivas Dande

* @Company: Java Learning Center

**/

```
class Solution {
    public int search(int[] arr, int element) {
```



```
int n = arr.length;
int low = 0;
int high = n - 1;

while (low <= high) {
    int mid = (low + high) / 2;

    if (element == arr[mid]) {
        return mid;
    }

    else if (arr[mid] >= arr[low]) {
        if (element >= arr[low] && element < arr[mid]) {
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }

    else {
        if (element <= arr[high] && element > arr[mid]) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
}

return -1;
}
```

162. Find Peak Element

Description:

- ✓ A peak element is an element that is strictly greater than its neighbors.
- ✓ Given a 0-indexed integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks.
- ✓ You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [1,2,3,1]` Output: 2

Example 2:

Input: `nums = [1,2,1,3,5,6,4]` Output: 5

Constraints:

- a) $1 \leq \text{nums.length} \leq 1000$
- b) $-231 \leq \text{nums}[i] \leq 231 - 1$
- c) $\text{nums}[i] \neq \text{nums}[i + 1]$ for all valid i .

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Amazon 5

Apple 2

Facebook 2



Java Learning Center

No.1 In Java Training & placement

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Google **3**

Microsoft **3**

Uber **3**

Adobe **2**

Companies

0 - 6 months

6 months - 1 year

1 year - 2 years

Roblox **12**

HRT **7**

Snapchat **4**

Bloomberg **3**

Walmart Labs **3**

Yahoo **2**

IXL **2**

VMware **2**

PayPal **2**

TikTok **2**

162. Find Peak Element

```
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
class Solution {
    public int findPeakElement(int[] arr) {
        int low = 0;
        int high = arr.length - 1;
        while (low < high) {
            int mid = (low+high) / 2;
            if (arr[mid] > arr[mid + 1])
                high = mid;
            else
                low = mid + 1;
        }
        return low;
    }
}
```